



PARTE 2 · PRUEBA PRÁCTICA

Prueba técnica de IA

Cinco ejercicios sobre un repositorio que corre en local. Take-home asíncrono, con una sesión de revisión posterior.

Duración 3-4 h

Entrega 72 h

Total 100 pts

Duración estimada: 4 horas de trabajo. **Entrega:** no hay fecha límite. **Entrégala en cuanto esté lista.** **Base de código:** carpeta `starter_kit/` adjunta. Autocontenida, sin credenciales. **Entregable:** repositorio Git (público o invitándonos), con `NOTAS.md` incluido.

Trabaja en **commits reales** conforme avanzas, no en un único commit al final. Revisamos el historial: dice más de cómo trabajas que el resultado final. Anota en `NOTAS.md` a qué hora empezaste y a qué hora entregaste.

Contexto

Vas a trabajar sobre el esqueleto de un **asistente de IA** integrado en una plataforma SaaS: los clientes suben su propia documentación y el asistente responde preguntas de sus usuarios usando ese contenido.

El pipeline tiene cinco pasos: **Clasificador** (detecta la intención de la pregunta) → **Recuperador** (busca fragmentos relevantes) → **Especialistas** (generan respuesta, en paralelo) → **Verificador** (revisa que esté sustentada) → **Redactor** (arma la respuesta final).

Los ejercicios tocan tres de esos pasos, dos problemas transversales que aparecen en cualquier sistema con LLMs en producción, y al final los conectas todos en algo que se puede abrir y usar.

No necesitas experiencia previa con ningún framework de agentes ni con un proveedor de nube específico. Todo corre en local con `pytest`.

Este documento es autocontenido: no necesitas nada de la entrevista previa para resolverlo. Si algo te parece ambiguo, decide, anótalo en `NOTAS.md` y sigue — así trabajamos también en el día a día.

Antes de empezar

1. Lee `starter_kit/README.md` y lee `shared/clients.py` completo. Casi todos los errores que vemos en esta prueba se evitan leyendo ese archivo.
2. Corre `pytest`. Van a fallar cosas. Ese es el punto de partida.
3. Levanta la consola con `python app.py` y ábrela en el navegador. Va a fallar, porque falta implementarla — pero así ves desde el minuto uno hacia dónde vas.
4. Trabaja en commits pequeños con mensajes reales.

Por dónde empezar si el tiempo aprieta

El **ejercicio 6 es el que más pesa** y depende de los ejercicios 1 y 2. Si tuvieras que elegir, ese es el camino: **1 → 2 → 6**, y después el resto. Una consola que funciona con dos ejercicios pendientes vale más que cinco ejercicios sueltos sin nada que abrir.

Lo que NO evaluamos

No nos importa si terminas los 6 ejercicios. Alguien que entrega 4 ejercicios impecables con su razonamiento documentado puntúa **por encima** de quien entrega 6 a medias. Si te quedas sin tiempo, escribe en `NOTAS.md` qué falta y cómo lo habrías resuelto.

Tampoco evaluamos diseño gráfico. La consola puede verse sobria; lo que se califica es que se entienda qué hizo el asistente y por qué.

Lo que **SÍ** evaluamos

Que el código funcione, que los tests prueben algo real, y sobre todo **que puedas explicar por qué tomaste cada decisión**. Habrá una sesión de revisión de 30 minutos donde te pediremos que expliques tu propio código y hagas un cambio en vivo.

| Ejercicio 1 — Clasificador de intención (15 pts)

Archivo: `config/intents.py` **Tests:** `tests/test_classify_intent.py` (ya escritos, no los modifiques)

Implementa `classify_intent(message) -> str`. Todo el contrato está en el docstring de la función: normalización de mayúsculas/acentos/signos, el patrón más largo gana, se ignoran las líneas citadas (`>`), longitud mínima, entradas vacías.

Criterio de aceptación: los 18 tests pasan.

Se evalúa:

7 Los tests pasan
pts

5 La solución es legible y no repite lógica (la normalización se escribe una vez)
pts

3 No hackeaste los casos: nada de `if message == "..."`. Si agregamos un patrón nuevo a `INTENTS`, tu código lo respeta
pts sin tocarse

Pista: `unicodedata.normalize("NFKD", s)` te resuelve acentos y ñ sin instalar nada.

Nota conceptual (va en `NOTAS.md`, máx. 120 palabras): esta clasificación es por reglas. En producción muchos equipos la harían con un LLM o con un clasificador entrenado. Da **una ventaja y una desventaja concretas** de las reglas frente al LLM para este caso, y di en qué situación cambiarías de enfoque.

| Ejercicio 2 — Reporte de cobertura (15 pts)

Archivo: `tools/intent_report_tool.py` **Tests:** `tests/test_intent_report.py` (uno dado, agregas ≥4)

Implementa `count_messages_by_intent(workspace_id) -> dict[str, int]` según su docstring: intenciones activas desde la base de datos, mensajes desde el almacenamiento, clasificación con el ejercicio 1, intenciones con conteo 0 incluidas, `desconocido` siempre presente.

Restricciones que se califican:

- Reutiliza `get_db_client()` y `get_storage_client()`. **Cero clientes nuevos dentro de la función.**
- La función es `async` y hace `await` de verdad.
- Decide y **documenta con un comentario** qué pasa si el workspace no existe. Cualquiera de las tres opciones (propagar / dict vacío / ceros) puede ser correcta; lo que no es correcto es no haberlo pensado.

Se evalúa:

5 pts	Conteos correctos, incluyendo intenciones en 0 y desconocido
3 pts	Filtra las intenciones inactivas leyéndolas de la base (no hardcodeadas)
3 pts	Respetar los singletons — hay un test tuyo que lo demuestra
2 pts	Tus tests cubren los casos pedidos y sus nombres explican qué protegen
2 pts	Manejo del workspace inexistente, decidido y justificado

Ejercicio 3 — Code review de código real (20 pts)

Archivo: `tools/legacy_answers_tool.py` **Entregable:** el archivo corregido + una sección en `NOTAS.md`

Ese código está en producción y "funciona". Tiene **al menos seis defectos** distintos, de gravedades muy diferentes: uno mezcla datos entre usuarios, otro tira el proceso completo, otro es una decisión que simplemente no sobrevive a un servicio sin estado.

Tu entrega:

1. Una tabla en `NOTAS.md` con una fila por defecto:

#	Línea	Qué está mal	Síntoma que produce en producción	Gravedad
---	-------	--------------	-----------------------------------	----------

La columna del **síntoma** es la que más pesa. "Está mal usar un mutable como valor por defecto" es la mitad de la respuesta; "el segundo usuario que consulta el workspace recibe los datos filtrados del primero" es la respuesta completa.

1. **El archivo corregido**, conservando el propósito de la función.
2. **Un test** que falle con el código original y pase con el tuyo, para el defecto que consideres más grave. Ese test es la prueba de que entendiste el bug y no solo lo maquillaste.

Se evalúa:

9 pts	Defectos encontrados (1.5 pts c/u, hasta 6)
5 pts	Los síntomas descritos son concretos y correctos
4 pts	La corrección es limpia y no introduce problemas nuevos
2 pts	El test de regresión falla con el original y pasa con el arreglo

Si encuentras más de seis, dilo. Suma.

Ejercicio 4 — Prompt del Verificador (15 pts)

Archivos: `prompts/verificador_v1.md` (actual) → creas `prompts/verificador_v2.md`

El Verificador es el paso que decide si una respuesta generada llega al usuario o se bloquea. El prompt actual (v1) es demasiado permisivo y deja pasar alucinaciones.

El incidente que hay que evitar. Un usuario preguntó cuántos días de garantía tiene el plan Pro. El asistente respondió, con total seguridad y muy bien redactado, que 30 días. La documentación del cliente dice 15. La búsqueda sí había traído fragmentos, pero ninguno hablaba de garantías del plan Pro; el modelo completó el dato por su cuenta, y la respuesta se entregó sin ninguna cita. El Verificador la aprobó porque su prompt solo pedía "que tenga una cita" — y la tenía.

Escribe la versión 2. Debe:

1. **Rechazar citas genéricas.** "según la documentación, sí se puede" no es una cita: es una paráfrasis vacía. Una cita es texto literal del contenido del cliente. Define el criterio de forma que el modelo pueda **aplicarlo**, no de forma abstracta.
2. **Introducir un tercer veredicto: DUDOSO.** Cuando la cita es real pero falta un metadato de trazabilidad (el índice del fragmento, por ejemplo), la respuesta no debe descartarse ni aprobarse: se marca para revisión. Explica en tu documento por qué "rechazar" sería la decisión equivocada de negocio.
3. **Usar el puntaje de similitud.** Cada respuesta trae un campo `similitud`. Define cómo entra en la decisión y justifica el umbral que elijas.
4. **Prohibir explícitamente que el Verificador modifique la respuesta.** Su trabajo es juzgar, no reescribir.
5. **Definir un formato de salida JSON estricto**, con un ejemplo de entrada y su salida esperada.

Además, entrega `prompts/casos_verificador.md`: las 5 respuestas de `fixtures/db.json` (a-1 a a-5), con el veredicto que tu prompt debería producir para cada una y una línea de por qué. Una de ellas tiene una trampa: encuéntrala.

Se evalúa:

4 pts	Los criterios son <i>accionables</i> — un modelo puede aplicarlos sin adivinar
3 pts	El esquema de salida es estricto y está ejemplificado
2 pts	El tratamiento de <code>DUDOSO</code> está bien argumentado en términos de negocio
2 pts	El umbral de similitud está justificado, no puesto al azar
4 pts	Los 5 casos, con la trampa detectada

No evaluamos prosa bonita. Evaluamos si ese prompt reduciría alucinaciones en producción. Un prompt de 20 líneas específicas gana a uno de 200 genéricas.

Nota conceptual (en `NOTAS.md`, máx. 120 palabras): un verificador que es otro LLM también puede equivocarse. ¿Qué parte de esta verificación moverías a código determinista en vez de dejarla en el prompt, y por qué esa parte y no las otras?

| Ejercicio 5 — Guardia anti-loop (10 pts)

Archivos: `tools/loop_guard.py` + `tests/test_loop_guard.py` (lo escribes completo)

Implementa `LoopGuard` según su docstring: conteo por `(sesión, agente)`, `ToolLoopError` al pasar de `MAX_CALLS`, mensaje de error accionable, aislamiento entre agentes y sesiones, `reset()` y `snapshot()`.

Escribe tú la suite de tests completa.

Se evalúa:

4 pts	Implementación correcta, incluyendo el aislamiento por sesión y agente
3 pts	Suite de tests: cubre los casos listados, sin tests de relleno
2 pts	El mensaje de error sirve para depurar a las 3 a.m.
1 pt	El comentario sobre concurrencia demuestra que lo pensaste

Y responde en `NOTAS.md`, en **máximo 150 palabras**: ¿dónde exactamente enchufarías esta guardia para que ningún agente pueda saltársela, y por qué ahí y no en cada herramienta?

Ejercicio 6 — La consola del asistente (25 pts)

Archivos: `app.py` + `tests/test_app.py` (dos tests dados, agregas ≥2) **Se levanta con:** `python app.py` → `http://localhost:8000`

Este es el ejercicio que más pesa, y el único que alguien puede *abrir y usar*. Aquí conectas las piezas anteriores en una consola que muestra, paso a paso, qué hizo el asistente con una pregunta y por qué.

El servidor y el esqueleto de la página **ya funcionan**: si corres `python app.py` ahora mismo, la página carga y te devuelve un error, porque falta `consultar()`. Eso es lo que construyes.

Parte A — El pipeline (`consultar`)

Implementa `async def consultar(pregunta, workspace_id) -> dict` según su docstring: clasifica la intención, recupera fragmentos con `shared.retriever`, y decide el veredicto según el mejor puntaje de similitud.

Veredicto	Cuándo	Qué se responde
<code>SIN_EVIDENCIA</code>	No hay fragmentos, o el mejor queda bajo <code>UMBRAL_MINIMO</code>	Nada. <code>respuesta</code> es <code>None</code>
<code>DUDOSO</code>	El mejor está entre <code>UMBRAL_MINIMO</code> y <code>UMBRAL_ALTO</code>	Se responde, marcado para revisión
<code>APROBADO</code>	El mejor llega a <code>UMBRAL_ALTO</code> o más	Se responde

El camino de `SIN_EVIDENCIA` es el importante. Es el que evita el incidente del ejercicio 4: el asistente prefiere callarse antes que responder algo que no puede respaldar.

En esta prueba **no hay modelo de lenguaje ni llamadas a ninguna API**: la "respuesta" es el texto literal del mejor fragmento. Así nada de lo que muestra la consola puede estar inventado, y todo corre sin claves ni internet.

`consultar()` debe ser **testable sin levantar el servidor**: recibe texto, devuelve un diccionario. Si tu lógica queda atrapada dentro del handler HTTP, no vas a poder escribir los tests.

Parte B — La página

Haz que un humano entienda de un vistazo qué pasó. Como mínimo debe verse:

- la **intención** detectada
- los **fragmentos** recuperados, cada uno con su puntaje de similitud
- el **veredicto**, visualmente distinto en los tres casos
- la **respuesta final**, o el aviso de que no hay evidencia suficiente

El criterio es uno solo: **debe quedar imposible confundir "esto está sustentado" con "esto no lo sé"**. Si un compañero abre tu consola sin haber leído nada, esa distinción tiene que saltarle a la cara.

Restricciones

- **Cero dependencias nuevas.** Todo sale de la librería estándar: nada de Flask, FastAPI, React, Tailwind ni fuentes externas. Una sola página servida desde el mismo archivo.
- No modifiques `shared/retriever.py` — te lo damos hecho.
- Los umbrales que vienen en `app.py` son un punto de partida. Si los cambias, di por qué en `NOTAS.md`.

Pruébalo con estas tres preguntas

Cada una cae en un veredicto distinto. Deben verse claramente diferentes:

Pregunta	Debe dar
<i>¿Cuántos días de garantía tiene el plan Pro?</i>	APROBADO
<i>olvidé mi contraseña, ¿cómo la restablezco?</i>	DUDOSO
<i>¿cuánto dura la garantía del plan Enterprise?</i>	SIN_EVIDENCIA

La tercera es la interesante: el corpus **no dice nada** del plan Enterprise, pero el recuperador igual devuelve fragmentos que hablan de planes y de garantías. Se ven bien. No responden la pregunta.

Se evalúa:

8 pts	<code>consultar()</code> correcta: los tres veredictos, con <code>respuesta: None</code> en <code>SIN_EVIDENCIA</code>
7 pts	La página comunica: se entiende el pipeline sin leer el código, y la abstención es inconfundible
4 pts	<code>consultar()</code> está separada del servidor y es testeable directamente
3 pts	Tus tests cubren <code>DUDOSO</code> y al menos un borde
3 pts	El <code>motivo</code> es accionable: cita números y umbrales, no "no se pudo responder"

Nota conceptual (en `NOTAS.md`, máx. 120 palabras): el recuperador que te dimos es léxico — compara palabras, no significado — y por eso falla con sinónimos. **No es tu trabajo arreglarlo.** Dinos: ¿qué cambiarías para que entienda que "restablecer contraseña" y "recuperar acceso" son lo mismo, y qué te costaría ese cambio?

NOTAS.md — obligatorio

Sin este archivo no calificamos la entrega. Estructura:

```
# Notas de entrega – <tu nombre>

## Resumen
Qué completé, qué no, y por qué.

## Tiempo
Hora en que empecé, hora en que entregué, y horas de trabajo real por ejercicio.
Aproximado y honesto.

## Decisiones
Las 3 decisiones de diseño de las que estoy más seguro, y por qué.
Las 2 de las que estoy menos seguro, y qué me haría cambiar de opinión.

## Ejercicio 1 – reglas vs LLM
(máx. 120 palabras)

## Ejercicio 3 – tabla de defectos
(la tabla pedida)

## Ejercicio 4 – qué movería a código determinista
(máx. 120 palabras)

## Ejercicio 6 – cómo mejoraría el recuperador
(máx. 120 palabras)

## Ejercicio 5 – dónde enchufo la guardia
(máx. 150 palabras)

## Uso de IA
Qué herramienta usé, en qué ejercicios y para qué exactamente.
"Copilot autocompletando" y "le pedí que escribiera el ejercicio 2" son
respuestas distintas y ambas aceptables – declararlas mal, no.

## Qué haría con una semana más
```

Sobre la IA: usarla no resta puntos. Ocultarla sí, y lo notamos en la sesión de revisión. Lo que evaluamos es tu criterio sobre lo que la IA produjo.

Cómo entregar

```
entrega_<tu_nombre>/
├── NOTAS.md
├── app.py                ← la consola
├── config/intents.py
├── tools/
│   ├── intent_report_tool.py
│   ├── legacy_answers_tool.py
│   └── loop_guard.py
├── prompts/
│   ├── verificador_v2.md
│   └── casos_verificador.md
└── tests/
    ├── test_app.py
    ├── test_intent_report.py
    ├── test_loop_guard.py
    └── (tu test de regresión del ejercicio 3)
```

Incluye en `NOTAS.md` una **captura de pantalla de tu consola** con la pregunta del plan Enterprise. Si por lo que sea no arranca en nuestra máquina, esa captura es lo que nos deja evaluarla igual.

Antes de enviar, verifica en un clon nuevo que `pytest` corre limpio y que `python app.py` levanta la consola. Una entrega que no arranca en la máquina del evaluador se califica sobre lo que se puede leer, no sobre lo que se puede correr.

Sesión de revisión (30 min, en vivo)

No es una trampa, es la parte más importante. El formato:

- **10 min** — abres tu consola y nos la enseñas funcionando, y nos explicas el ejercicio que más te costó.
- **10 min** — cambio en vivo: te pedimos una modificación pequeña sobre tu propio código. Buscamos que te muevas cómodo en lo que escribiste.
- **10 min** — preguntas sobre tus decisiones y tus dudas. Aquí es donde "no lo sé, pero lo averiguaría así" suma puntos.

Puntaje total

Ejercicio	Puntos
1 — Clasificador de intención	15
2 — Reporte de cobertura	15
3 — Code review	20
4 — Prompt del Verificador	15
5 — Guardia anti-loop	10
6 — La consola del asistente	25
Total	100

Nota final del proceso: 25% Parte 1 (hoja teórica) + 75% Parte 2.

La hoja teórica de 15 minutos es un punto de partida, no un veredicto: pesa poco a propósito. Esta parte es la que decide.

Nota final	Resultado
85–100	Avanza al final del proceso
70–84	Avanza, con un plan de acompañamiento de 4 semanas
55–69	Segunda conversación técnica antes de decidir
< 55	No continúa en este momento